Movie Recommendation System

AI Laboratory — Project Report

Zeynep Eskicioglu Valentina Macek

1 Introduction

Choosing a movie from a catalog of thousands of titles takes time. Streaming platforms solve this with recommendation systems, and we wanted to understand how these systems actually work by building one ourselves.

Our project is a web app that recommends movies in different ways. A user can search for a movie they like and get similar ones, get personal recommendations based on a rating history, or combine both approaches. We also handle the case where the user is completely new and the system knows nothing about them.

The two main techniques we use are content-based filtering ("find movies similar to this one") and collaborative filtering ("find movies that users with similar taste enjoyed"). On top of these we built a hybrid model that mixes both signals, and a separate cold-start model for new users.

2 Solution

2.1 Data

We use two datasets, because no single free dataset gave us both rich movie information and real user ratings:

- **TMDB movies dataset (Kaggle), ∼238,000 movies.** It contains title, genres, plot overview, cast, director, producers and writers. We use it for content-based filtering, since it describes what each movie is about.
- **MovieLens 100K (GroupLens), ∼100,000 ratings** from 610 users on ∼9,700 movies. We use it for collaborative filtering, since it tells us what real users actually liked.

Preprocessing was mostly about cleaning text fields: filling missing values, normalising titles (lowercase, removing punctuation) so that lookups are tolerant to typos, and combining the text columns of each TMDB movie into a single "features" string for vectorisation.

2.2 Models

Content-based filtering. We turn the combined description of each movie into a TF-IDF vector (limited to the 50,000 most informative terms) and rank movies by cosine similarity to the query movie. Our first version precomputed the full similarity matrix, which crashed: a 238K × 238K matrix does not fit in memory. The fix was to compute similarities on the fly, only for the one movie the user asked about. This made the app fast enough and removed the memory problem.

Collaborative Filtering (SVD). We use Singular Value Decomposition (SVD) for matrix factorisation via the `surprise` library (50 latent factors, 20 epochs). SVD factorises the rating matrix into two latent factor matrices U (users) and V (items), capturing hidden taste dimensions such as genre preference or narrative style. The `surprise` implementation follows the Simon Funk (2006) SGD-based formulation, which never explicitly constructs the full matrix — it iterates only over

observed ratings, ignoring missing entries entirely rather than imputing them. At inference time, the predicted rating for any unseen user(u)–movie(i) pair is:

$$\hat{r}_{ui} = \mu + b_u + b_i + U_u \cdot V_i^\top \tag{1}$$

where μ is the global mean, b_u and b_i are bias terms, and $U_u \cdot V_i^\top$ is the dot product of the learned latent vectors. The top-n highest predicted ratings for unwatched movies become the recommendations.

Hybrid model. Content-based ignores personal taste, and collaborative filtering ignores the movie you just searched for. The hybrid model answers a more useful question: "which movies like this one would I personally enjoy?" It normalises both scores to $[0, 1]$ and combines them as

$$\text{score} = \alpha \cdot \text{content} + (1 - \alpha) \cdot \text{CF}.$$

The user controls α with a slider in the app, so $\alpha = 1$ is pure content and $\alpha = 0$ is pure collaborative filtering. Since the two datasets only partially overlap, a movie that exists in only one of them keeps its single signal instead of being dropped.

New-user model (cold start). SVD cannot predict anything for a user who was not in the training data. To handle new users, we ask them to rate at least three movies, then find the 20 most similar existing users with Pearson correlation on the shared ratings, and take a weighted average of their ratings as the collaborative score. This is combined with the content signal from the movies the new user liked, in the same way as the hybrid model.

2.3 Architecture

We split the project into a Flask backend and a Streamlit frontend. The backend loads the models and exposes one REST endpoint per recommendation type; the frontend only sends requests and displays results. We made this split so we could work in parallel without touching each other's code, and so the model logic stays independent from how it is displayed.

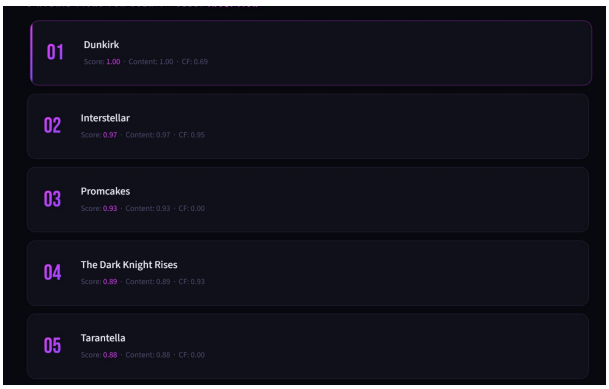
Each endpoint answers one clear question: `/content` ("which movies are similar to this one?"), `/collaborative` ("what should I watch next, based on my history?"), `/hybrid` ("which movies like this one would I like?"), `/new_user` ("what should I watch, given the few movies I rated?"), plus helper endpoints for genre browsing and for picking seed movies to rate.

3 Results

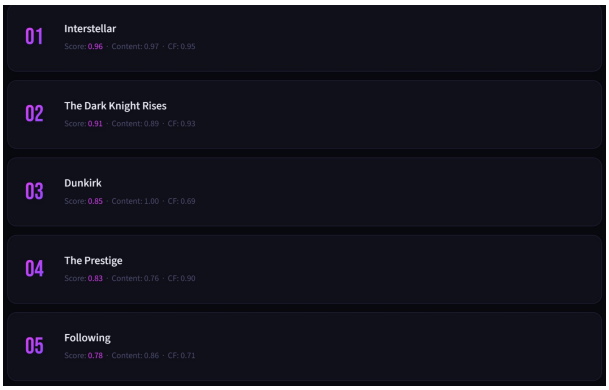
Collaborative filtering. We evaluated the SVD model with 5-fold cross-validation on the MovieLens ratings. The mean RMSE was 0.8699 and the mean MAE was 0.6679, with very small variation across folds (RMSE standard deviation 0.0033). On a 0.5–5.0 rating scale this means the predictions are on average less than one star away from the true rating, which we consider acceptable for a dataset of this size.

Content-based filtering. There is no ground truth for "similarity", so a numeric metric like RMSE does not apply here. Instead we checked the results qualitatively with known examples. For instance, querying "Inception" returns *Interstellar*, *Dunkirk*, *The Dark Knight Rises* and *Tarantella* — all films by the same director with overlapping cast and themes, which is exactly the behaviour we wanted.

Hybrid model. We tested the effect of α by keeping the same user and seed movie ("Inception", user 1) and moving the slider. With $\alpha = 1$ (pure content), movies like *Promcakes* and *Tarantella* appear in the top 5 despite having a CF score of 0.00 — they are textually similar but have no rating data. When α is lowered, these disappear and are replaced by *The Prestige* (CF: 0.90) and *Following* (CF: 0.71), which the user is predicted to enjoy. Figure 1 shows both cases side by side.



$\alpha = 1$ (pure content)



$\alpha = 0.5$ (balanced)

Figure 1: Hybrid recommendations for "Inception" (user 1) at two different α values. Lowering α removes content-only matches and promotes movies the user is predicted to like.

4 Limitations and reflections

- **Scale of the similarity computation.** The memory crash with the full similarity matrix taught us to think about memory before writing code. Computing on the fly works for one query at a time, but a real system with many users would need something smarter, like approximate nearest-neighbour search.
- **Dataset overlap.** The TMDB and MovieLens datasets are joined purely on title and release year, as no shared identifier (e.g. IMDb ID) is available in both. Despite applying title normalisation and a ± 1 year tolerance, 11.5% of MovieLens movies could not be matched to a TMDB entry and therefore receive only the CF signal in the hybrid model. A cleaner solution would be to use a dataset that provides a common identifier across both sources.
- **Imperfect similarity results.** TF-IDF only looks at word overlap, so an obscure movie with a similar plot description can rank above a more sensible recommendation. Some odd titles appear among the top results for popular queries.
- **Old and small ratings data.** MovieLens 100K has only 610 users and few recent movies. The collaborative model can only recommend movies that exist in this dataset.
- **Title matching.** Our partial title matching helps with typos but can occasionally pick the wrong movie when titles are substrings of each other.

5 Sources and contributions

Datasets: TMDB movies dataset (Kaggle); MovieLens 100K (GroupLens, University of Minnesota).

Libraries and documentation: scikit-learn (TF-IDF, cosine similarity), scikit-surprise (SVD), pandas, NumPy, Flask, Streamlit. We also used AI tools for debugging and for explanations of library functions; all code was reviewed, tested and adapted by us.

Contributions: Valentina Macek implemented the Flask backend (API routes, validation, configuration). Zeynep Eskicioglu built the Streamlit frontend (pages, components, styling). The models we developed together, Zeynep focused on collaborative filtering model, while Valentina focused on content-based model and the idea of hybrid model.